

Unit 4

Deep Learning

What is Data Augmentation? How Does Data Augmentation Work for Images?

In the field of machine learning, especially within the domain of deep learning, data plays a pivotal role in training effective models. However, collecting and annotating large datasets can be both time-consuming and expensive. Data augmentation offers a solution to this problem by artificially expanding the size and variability of a dataset. This technique is particularly beneficial in image processing tasks, where it helps improve the robustness and generalization capability of models.

Understanding Data Augmentation

Data augmentation is a technique used to increase the diversity of a dataset without actually collecting new data. By applying various transformations to the existing data, we can create new, modified versions that help the model learn to generalize better. These transformations are applied in a way that maintains the original label of the data, ensuring that the augmented data remains useful for training.

Importance of Data Augmentation

Data augmentation is essential for several reasons:

- **Improves Model Generalization:** By exposing the model to a wider variety of data, it learns to generalize better to unseen data.
- **Reduces Overfitting:** Augmented data helps prevent the model from learning noise and memorizing the training data.
- **Enhances Robustness:** Models trained with augmented data are more robust to variations and distortions in real-world data.
- **Cost-Effective:** It reduces the need for collecting and annotating large amounts of new data.

How Does Data Augmentation Work for Images?

Data augmentation for images involves applying various transformations to the original images to create new training examples. These transformations can be broadly categorized into several types:

1. Geometric Transformations

Geometric transformations alter the spatial properties of an image. Common geometric transformations include:

- **Rotation:** Rotating the image by a certain angle (e.g., 90°, 180°).
- **Flipping:** Flipping the image horizontally or vertically.
- **Scaling:** Zooming in or out of the image.

- Translation: Shifting the image along the x or y axis.
- Shearing: Slanting the shape of the image.

2. Color Space Augmentations

Color space augmentations modify the color properties of an image. These include:

- Brightness Adjustment: Increasing or decreasing the brightness of the image.
- Contrast Adjustment: Changing the contrast to make the image appear more or less vivid.
- Saturation Adjustment: Modifying the intensity of colors in the image.
- Hue Adjustment: Shifting the colors by changing the hue.

3. Kernel Filters

Kernel filters apply convolutional operations to enhance or suppress specific features in the image. Examples include:

- Blurring: Applying Gaussian blur to smooth the image.
- Sharpening: Enhancing the edges to make the image sharper.
- Edge Detection: Highlighting the edges in the image using filters like Sobel or Laplacian.

4. Random Erasing

Random erasing involves randomly masking out a rectangular region of the image. This helps the model become invariant to occlusions and improves its ability to handle missing parts of objects.

5. Combining Augmentations

Often, multiple augmentation techniques are combined to create more varied training data. For example, an image might be rotated, flipped, and then have its brightness adjusted in a single augmentation pipeline.

Tools and Libraries for Image Data Augmentation

Several tools and libraries facilitate image data augmentation:

- TensorFlow: TensorFlow's `tf.image` module provides functions for image transformations.
- Keras: Keras offers the `ImageDataGenerator` class for real-time data augmentation.
- PyTorch: PyTorch's `torchvision.transforms` module includes a wide range of augmentation techniques.
- Albumentations: A fast image augmentation library with a rich set of transformations.
- imgaug: A flexible library for image augmentation with support for various augmentations.

Noise robustness in deep learning

Noise robustness in deep learning refers to a model's ability to maintain accuracy and reliability even when presented with noisy or corrupted input data or labels. This is crucial because real-world data often contains imperfections and errors.

Why is Noise Robustness Important?

Real-World Data:

Real-world data is rarely perfect. It can contain errors, inconsistencies, and variations that can negatively impact the performance of deep learning models.

Adversarial Attacks:

Malicious actors can intentionally introduce noise or perturbations to input data to mislead deep learning models, posing a significant security risk.

Generalization:

A robust model is more likely to generalize well to unseen data, even if it's noisy, making it more reliable in real-world applications.

Applications:

Noise robustness is particularly important in applications where the consequences of incorrect predictions can be severe, such as medical diagnosis, autonomous driving, and financial analysis.

Methods to Enhance Noise Robustness:

Data Augmentation:

Adding Noise: Intentionally adding noise to training data can help models learn to be more resilient to variations and errors.

Alpha-Stable Noise: Using alpha-stable noise, which is more representative of real-world noise, can improve robustness to impulsive noise compared to traditional Gaussian noise.

Color Perturbation: Using color perturbation based on image characteristics can help models become more robust against adversarial attacks.

Model Architecture:

Simpler Models: Simpler models may be less prone to overfitting and can be more robust to noise.

Stability Training: Stability training can lead to more robust features, as the distribution of feature distances becomes more concentrated towards 0.

Regularization:

Weight Decay: Regularization techniques like weight decay can help prevent overfitting and improve generalization.

Noise-Aware Training:

Noisy Label Learning: Developing algorithms that can handle noisy labels in training data is crucial.

Gradient Noise: Introducing gradient noise during training can help models become more robust to noise.

Data Abstraction:

Mitigating Noise Through Abstraction: Using data abstractions to reduce the impact of noise on model performance can be an effective approach.

Interpretability:

Understanding Adversarial Examples: Analyzing the datapaths of adversarial examples can help understand why they are misclassified and develop better defenses.

Focus on Quality and Quantity of Data:

Large and Diverse Datasets: Using large and diverse datasets can help models learn to generalize better and become more robust to variations and noise sources.

Hyperparameter Tuning:

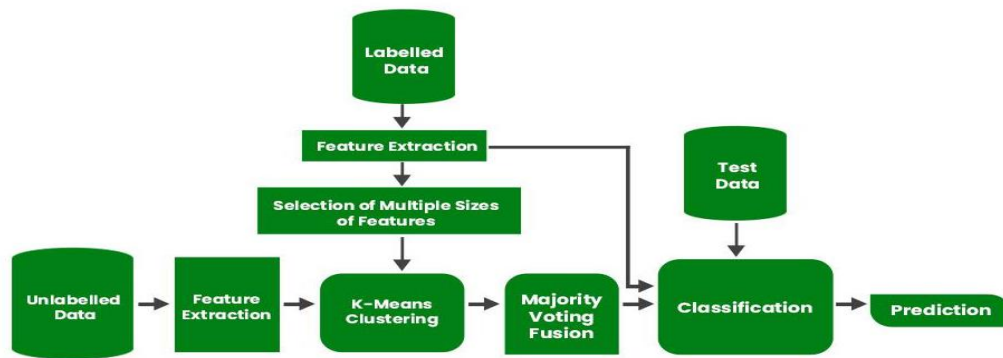
Optimal Parameter Settings: Finding the optimal settings for hyperparameters like learning rate and batch size can improve both model performance and robustness.

Examples of Noise in Different Domains:

- Image Recognition: Noisy images due to poor lighting, camera shake, or sensor issues.
- Audio Processing: Noisy audio recordings due to background noise, microphone quality, or transmission issues.
- Natural Language Processing: Noisy text data due to typos, grammatical errors, or inconsistent formatting.
- Medical Diagnostics: Noisy signals from medical sensors or images.
- Financial Data: Noisy data due to errors in data entry or sensor malfunctions.

What is Semi-Supervised Learning?

Semi-supervised learning is a type of machine learning that falls in between supervised and unsupervised learning. It is a method that uses a small amount of labeled data and a large amount of unlabeled data to train a model. The goal of semi-supervised learning is to learn a function that can accurately predict the output variable based on the input variables, similar to supervised learning. However, unlike supervised learning, the algorithm is trained on a dataset that contains both labeled and unlabeled data.



Examples of Semi-Supervised Learning

- Text classification: In text classification, the goal is to classify a given text into one or more predefined categories. Semi-supervised learning can be used to train a text classification model using a small amount of labeled data and a large amount of unlabeled text data.
- Image classification: In image classification, the goal is to classify a given image into one or more predefined categories. Semi-supervised learning can be used to train an image classification model using a small amount of labeled data and a large amount of unlabeled image data.
- Anomaly detection: In anomaly detection, the goal is to detect patterns or observations that are unusual or different from the norm

Applications of Semi-Supervised Learning

- Speech Analysis: Since labeling audio files is a very intensive task, Semi-Supervised learning is a very natural approach to solve this problem.
- Internet Content Classification: Labeling each webpage is an impractical and unfeasible process and thus uses Semi-Supervised learning algorithms. Even the Google search algorithm uses a variant of Semi-Supervised learning to rank the relevance of a webpage for a given query.
- Protein Sequence Classification: Since DNA strands are typically very large in size, the rise of Semi-Supervised learning has been imminent in this field.

Multi-Task Learning(MTL) for Deep Learning

Multi-Task Learning (MTL) is a type of machine learning technique where a model is trained to perform multiple tasks simultaneously. In deep learning, MTL refers to training a neural network to perform multiple tasks by sharing some of the network's layers and parameters across tasks.

In MTL, the goal is to improve the generalization performance of the model by leveraging the information shared across tasks. By sharing some of the network's parameters, the model can

learn a more efficient and compact representation of the data, which can be beneficial when the tasks are related or have some commonalities.

There are different ways to implement MTL in deep learning, but the most common approach is to use a shared feature extractor and multiple task-specific heads. The shared feature extractor is a part of the network that is shared across tasks and is used to extract features from the input data. The task-specific heads are used to make predictions for each task and are typically connected to the shared feature extractor.

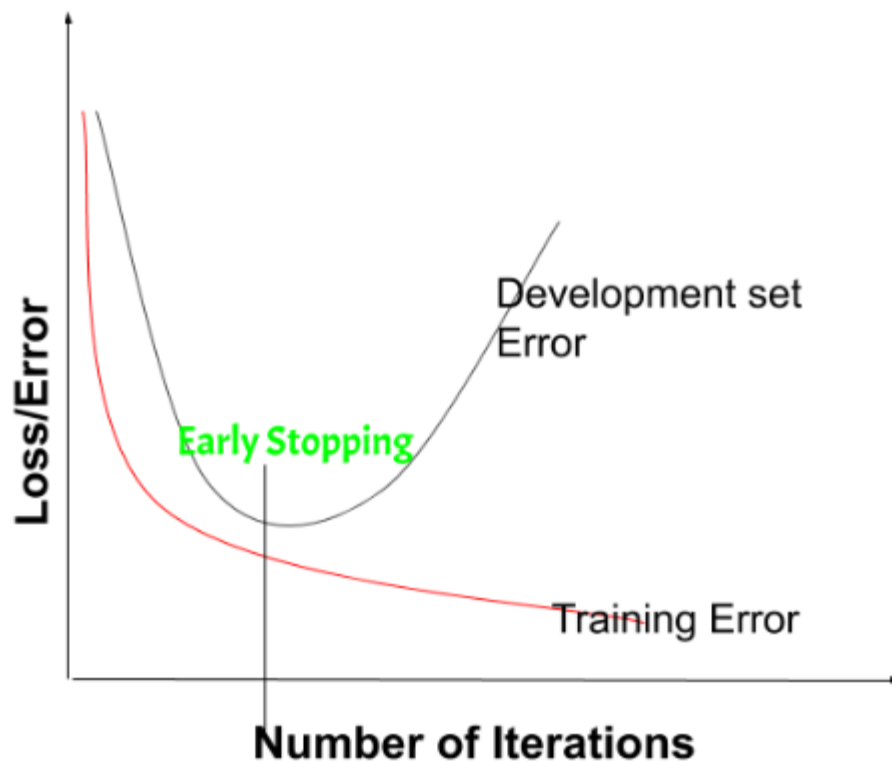
Another approach is to use a shared decision-making layer, where the decision-making layer is shared across tasks, and the task-specific layers are connected to the shared decision-making layer.

MTL can be useful in many applications such as natural language processing, computer vision, and healthcare, where multiple tasks are related or have some commonalities. It is also useful when the data is limited, MTL can help to improve the generalization performance of the model by leveraging the information shared across tasks.

What is Early Stopping?

In Regularization by Early Stopping, we stop training the model when the performance on the validation set is getting worse- increasing loss decreasing accuracy, or poorer scores of the scoring metric. By plotting the error on the training dataset and the validation dataset together, both the errors decrease with a number of iterations until the point where the model starts to overfit. After this point, the training error still decreases but the validation error increases.

So, even if training is continued after this point, early stopping essentially returns the set of parameters that were used at this point and so is equivalent to stopping training at that point. So, the final parameters returned will enable the model to have low variance and better generalization. The model at the time the training is stopped will have a better generalization performance than the model with the least training error.



Benefits of Early Stopping:

- Helps in reducing overfitting
- It improves generalisation
- It requires less amount of training data
- Takes less time compared to other regularisation models
- It is simple to implement

Parameter Sharing and Typing in Deep Learning

We usually apply limitations or penalties to parameters in relation to a fixed region or point. L2 regularisation (or weight decay) penalises model parameters that deviate from a fixed value of zero,

Parameter Typing

Two models are doing the same classification task (with the same set of classes), but their input distributions are somewhat different.

- We have model **A** has the parameters $w^{(A)}$
- Another model **B** has the parameters $w^{(B)}$

$$\hat{y}^{(A)} = f(w^{(A)}, x)$$

and

$$\hat{y}^{(B)} = g(w^{(B)}, x)$$

are the two models that transfer the input to two different but related outputs.

Parameter Sharing

The parameters of one model, trained as a classifier in a supervised paradigm, were regularised to be close to the parameters of another model, trained in an unsupervised paradigm, using this method (to capture the distribution of the observed input data). Many of the parameters in the classifier model might be linked with similar parameters in the unsupervised model thanks to the designs. While a parameter norm penalty is one technique to require sets of parameters to be equal, constraints are a more prevalent way to regularise parameters to be close to one another. Because we view the numerous models or model components as sharing a unique set of parameters, this form of regularisation is commonly referred to as parameter sharing. The fact that only a subset of the parameters (the unique set) needs to be retained in memory is a significant advantage of parameter sharing over regularising the parameters to be close (through a norm penalty). This can result in a large reduction in the memory footprint of certain models, such as the convolutional neural network.

Convolutional neural networks (CNNs) used in computer vision are by far the most widespread and extensive usage of parameter sharing. Many statistical features of natural images are translation insensitive. A shot of a cat, for example, can be translated one pixel to the right and still be a shot of a cat. By sharing parameters across several picture locations, CNNs take this property into account. Different locations in the input are computed with the same feature (a hidden unit with the same weights). This indicates that whether the cat appears in column i or column $i + 1$ in the image, we can find it with the same cat detector.

CNN's have been able to reduce the number of unique model parameters and raise network sizes greatly without requiring a comparable increase in training data thanks to parameter sharing. It's still one of the best illustrations of how domain knowledge can be efficiently integrated into the network architecture.

Bagging vs Boosting in Deep Learning

Ensemble learning helps improve machine learning results by combining several models. This approach allows the production of better predictive performance compared to a single model.

Basic idea is to learn a set of classifiers (experts) and to allow them to vote. **Bagging and Boosting are two types of Ensemble Learning.** These two decrease the variance of a single estimate as they combine several estimates from different models. So the result may be a model with higher stability. Let's understand these two terms in a glimpse.

- Bagging: It is a homogeneous weak learners' model that learns from each other independently in parallel and combines them for determining the model average.
- Boosting: It is also a homogeneous weak learners' model but works differently from Bagging. In this model, learners learn sequentially and adaptively to improve model predictions of a learning algorithm.

Bagging

Bootstrap Aggregating, also known as bagging, is a machine learning ensemble meta-algorithm designed to improve the stability and accuracy of machine learning algorithms used in statistical classification and regression. It decreases the variance and helps to avoid overfitting. It is usually applied to decision tree methods. Bagging is a special case of the model averaging approach.

Description of the Technique

Suppose a set D of d tuples, at each iteration i , a training set D_i of d tuples is selected via row sampling with a replacement method (i.e., there can be repetitive elements from different d tuples) from D (i.e., bootstrap). Then a classifier model M_i is learned for each training set $D < i$. Each classifier M_i returns its class prediction. The bagged classifier M^* counts the votes and assigns the class with the most votes to X (unknown sample).

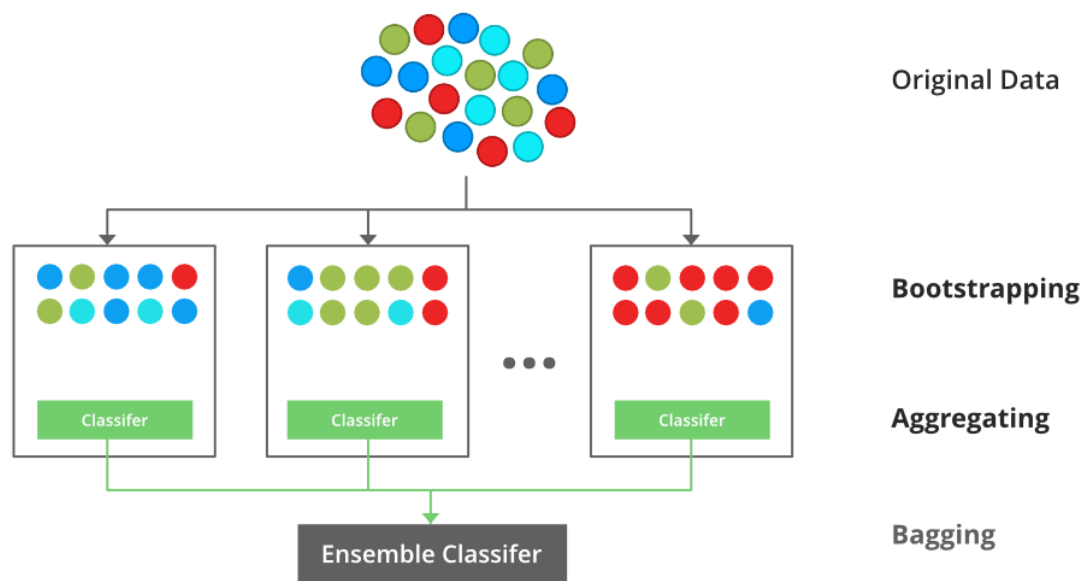
Implementation Steps of Bagging

Step 1: Multiple subsets are created from the original data set with equal tuples, selecting observations with replacement.

Step 2: A base model is created on each of these subsets.

Step 3: Each model is learned in parallel with each training set and independent of each other.

Step 4: The final predictions are determined by combining the predictions from all the models.



Example of Bagging

The Random Forest model uses Bagging, where decision tree models with higher variance are present. It makes random feature selection to grow trees. Several random trees make a Random Forest.